Finance overlays and audited overrides

How a closed, audited period is reconciled loan by loan from the system's own figures to the audited ones, as built and applied on 17 October 2026

17 October 2026

Edward Mazibuko, Director, Data Analytics, Dupleix Institute

For the development collaborators (Dupleix, Codex, Copilot reviews), MAIIC Finance, Credit and ICT, and Dr Thomson Kumwenda (CFO)

Built, tested and applied to December 2025. Supersedes version 1 of the same date, which proposed overlays only and was written before the decision in section 2.

Contents

1. What this document is about . . . 2

2. The decision this rests on . . . 2

3. Overlays: a record beside the figure, never a change to it . . . 2

4. Overrides: the governed decision that changes a booked figure . . . 2

4.1 The two kinds . . . 2

4.2 Sources of a staging override . . . 3

4.3 Rules every override follows . . . 3

4.4 Commands . . . 3

5. Three things that went wrong while building this, and what they teach . . . 4

6. What was applied for December 2025 . . . 4

6.1 Staging: 36 loans . . . 4

6.2 Gross amount: one loan . . . 4

6.3 Where it was applied, and how . . . 4

7. Reproducing it from nothing . . . 5

8. Tests . . . 5

9. What is still owed . . . 5

1. What this document is about

The system rebuilds every month's loan book from E-Banker's raw data. For December 2025, MAIIC's audited ECL workbook disagrees with that rebuild on 36 loans' arrears and on one loan's balance. This document describes the two tools now in the system for that gap, what was done with them for December 2025, and what is still owed.

- **Overlays** record Finance's view beside the system's figure and change nothing the system calculates.
- **Overrides** are governed, audit-logged decisions that replace the system's figure with the audited one, for one loan in one reporting period, when that period is built.

Version 1 of this document (same date) proposed overlays only and recommended that no override should touch a booked figure until a separate decision was taken. That decision was taken the same day (section 2), so the overrides described here were built. Version 1 is kept for the history of the reasoning.

2. The decision this rests on

On 17 October 2026 Edward Mazibuko decided as follows. December 2025 is a closed period whose audit has passed, and the auditors approved the workbook's loan-by-loan breakdown. MAIIC will supply the reasons for each difference by Monday 19 October 2026. Until then, the audit's approval is the reason. The overrides are the audit-logged way of carrying the system's figures across to the audited ones, so the step from system report to audited report is visible, not hidden. The original system values are kept and named on every row.

This is not a change to the staging policy or to how balances are built. Every loan without an approved override is built exactly as before.

3. Overlays: a record beside the figure, never a change to it

	Ageing overlay	Gross amount overlay
Service	`App\Services\Eir\LoanAgeingOverlayService`	`App\Services\Eir\LoanGrossAmountOverlayService`
Command	`php artisan eir:import-ageing-overlay {file} --user=`	`php artisan eir:import-gross-amount-overlay {file} --user=`
Table	`loan_ageing_overlays`	`loan_gross_amount_overlays`
December 2025 file	`docs/bootstrap/models/Finance Ageing Overlay - December 2025.csv` (36 rows)	`docs/bootstrap/models/Finance Gross Amount Overlay - December 2025.csv` (107 rows)

Both files are backfilled from the system's own figures, so Finance never retypes a known number. Every row's comment begins with the system's own value in words: "System bracket: 1-30 Days (12 days)." or "System amount: 138,135,680.80." The comment can never be read without the figure it answers. A later import for the same loan and period replaces the earlier one. Overlays feed no calculation. They are the evidence an override points to.

4. Overrides: the governed decision that changes a booked figure

4.1 The two kinds

	Staging basis override	Gross amount override
What it replaces	The day count staging reads (`staging_dpd` and every day count in `build_basis.dpd_counts`)	`carrying_amount`, which EAD and ECL read
What it keeps	`overdue_days` (the directive's own count) and the policy basis, named in `staging_dpd_basis`	`stored_carrying_amount`, the system's own figure
Service	`LoanStagingBasisOverrideService`	`LoanGrossAmountOverrideService`
Command	`php artisan eir:staging-basis-override`	`php artisan eir:gross-amount-override`
Table	`loan_staging_basis_overrides`	`loan_gross_amount_overrides`
Figure carried	`override_days` and `source` (see 4.2)	`override_amount`

4.2 Sources of a staging override

- `directive_count` (no day count given): use the schedule-aware count the build already computes for the RBM directive (`overdue_days`) instead of the staging policy's bucket reading. This is for a loan where that count is evidenced as right.
- `audited_workbook` (a day count given): an explicit day count that puts the loan in the audited bracket for a closed, audited period. For December 2025 this is the lower bound of the workbook's bracket (0, 31, 91 or 181 days), except where an independent reconstruction found the exact count (section 6).

4.3 Rules every override follows

1. **One reporting period.** An override names the month it reconciles (`reporting_period`, YYYY-MM), and only that month's build applies it. Proposing one needs `--period`.
2. **Maker-checker.** One person proposes, a different person approves. Self-approval works only under UAT mode (`php artisan governance:uat-mode on`), on a non-production server, for an administrator. It is recorded as "Maker-Checker Override Used" and appears on the self-approvals report.
3. **Audit-logged.** Proposed, approved, rejected and withdrawn are each logged with the person, the time and the figures.
4. **One live override per loan per period.** A loan can hold one PROPOSED or APPROVED override for a period. A withdrawn or rejected one stays on the record, so `contract_id` is indexed, not unique.
5. **Applied only at build.** Approving changes nothing until that month is built again with `eir:build-loan-books`. The build reads the approved overrides for the month it is writing.
6. **Named on the row.** The build adds a flag naming the decision number, the audited figure, the system's own figure and the reason. `build_flag` was widened to TEXT so a long audited reason is never cut short.

4.4 Commands

Action	Command
Propose a staging override	`php artisan eir:staging-basis-override 104430000087 --propose --period=2025-12 --reason="..." --user=1`
Propose a gross amount override	`php artisan eir:gross-amount-override 104420000005 --propose --period=2025-12 --amount=0 --reason="..." --user=1`
Approve (a different person)	`php artisan eir:staging-basis-override --approve=12 --user=2`
Reject, with a reason	`php artisan eir:gross-amount-override --reject=3 --reason="..." --user=2`
Withdraw	`php artisan eir:staging-basis-override --withdraw=12 --user=1`
List every override and its period	`php artisan eir:gross-amount-override --list`

The staging command proposes a `directive_count` override. An `audited_workbook` override, which carries a day count, is proposed through the service or the apply script (section 6.3).

5. Three things that went wrong while building this, and what they teach

These are recorded so no one has to rediscover them.

1. **A staging override that only set `staging_dpd` had no effect.** `StagingService::evaluate()` recomputes the day count from `build_basis.dpd_counts` under the staging policy in force, and does so whenever those counts are present, which is always. The first build applied 36 approved overrides, and `staging:december-check` still showed all 36 differing. The override now also sets all five counts (`oldest`, `oldest_recounted`, `bucket`, `bucket_recounted`, `since_maturity`). A test asserts this. Lesson: before overriding a stored figure, find every place that recomputes it instead of reading it. The gross amount override was checked this way: `EclExposure` reads `carrying_amount` directly and nothing recomputes it.
2. **A hard unique constraint on `contract_id`** stopped a loan being proposed again after its first override was withdrawn. It was dropped (migration `2026_10_17_150000`) in favour of rule 4.
3. **Overrides first applied to every month built.** Without a period, the first fresh bootstrap applied the December 2025 overrides to all 27 months from July 2024 to September 2026: MUSECO was zeroed every month, and 44 golden checks failed. Migration `2026_10_17_170000` added `reporting_period` and dated every existing override 2025-12, which is true of all of them. A test proves an override for one month leaves another untouched.

6. What was applied for December 2025

6.1 Staging: 36 loans

These are the 36 facilities whose arrears bracket differed from the audited workbook (December 2025 Loan Book Investigation Findings v1).

- **Two are evidenced:** JAT (104430000087) at 69 days and BenCal (104460000059) at 0 days. Independent ledger and schedule reconstructions by Codex and Copilot match the workbook for both.
- **The other 34 are set to the lower bound of the workbook's bracket.** Their reason is the audit's approval, with MAIIC's facility-level reasons due by 19 October 2026.

Result: `staging:december-check` reports 107 of 107 matched facilities in the same stage as the workbook, where 36 differed before.

6.2 Gross amount: one loan

MUSECO (104420000005): the system carries K138,135,680.80, and the audited workbook shows nil. The override sets the booked carrying amount to nil. The system's figure stays on `stored_carrying_amount`. Its EAD and ECL become nil. Every other facility in the gross amount file is within K1,000 of the workbook and has no override.

Because this removes a real exposure from the book rather than relabelling a bracket, it was put to Edward as its own question before it was applied, and he confirmed it. MAIIC's reason for this loan is the most important of the ones owed.

6.3 Where it was applied, and how

Database	What it holds
`maiic_ifrs9_candidate_20261017`	Bootstrapped 17 October; the 36 staging overrides and MUSECO; December rebuilt
`maiic_ifrs9` (production copy)	Migrated and given December 2025 the same day (see the Production Copy Upgrade and Hosting Runbook v1); the same 37 overrides; December check 107 of 107

Both were applied by Edward from the console with two scripts in the project root:

- `apply_staging_overrides.php` holds the 36 contracts, their workbook brackets and the two evidenced counts.
- `apply_gross_amount_overrides.php` reads the gross amount CSV and overrides every row more than K1,000 from the workbook.

Each script writes to the database `DB_DATABASE` names and prints that name first. They propose and approve as user 1 under UAT mode, so every one of the 37 approvals is a recorded self-approval. UAT mode was switched off on `maiic_ifrs9` afterwards.

7. Reproducing it from nothing

The overrides are people's decisions, so no client input file can rebuild them. They travel in the **decisions pack** (Staging Module Specification v1, section 8.10). The pack now carries two more files, `staging_overrides.json` and `gross_amount_overrides.json`. Each lists every override with its period, figures, reason, status, people (by e-mail) and dates. The replay puts them back before the loan book is built, in the same step as the account exclusions, so the build applies them exactly as the source did.

- **Pack:** `docs/bootstrap/decisions/2026-10-17-december-2025-audited`, exported from `maiic_ifrs9`. It holds 36 staging overrides, 1 gross amount override and the one user who made them.
- **Command:** `php artisan --env=bootstrap eir:bootstrap --fresh --force-wipe --with-client-inputs --build --run-engines --verify --with-decisions=2026-10-17-december-2025-audited`
- **Tie-out:** the replay reports approved overrides here against the source for each kind, and lists any it could not carry with the reason. A carried override keeps its original people and dates, and writes one audit entry naming the pack.

8. Tests

- `tests/Feature/Eir/LoanStagingBasisOverrideServiceTest.php` and `LoanGrossAmountOverrideServiceTest.php` cover maker-checker, reasons, reject, withdraw, re-proposal after withdrawal, and one live override at a time.
- `tests/Feature/Ebanker/StagingBasisOverrideBuildTest.php` and `GrossAmountOverrideBuildTest.php` cover: - the override reaching the booked row, including `dpd_counts`; - withdrawal; - no effect on another loan; - no effect on another month.
- `tests/Feature/Eir/LoanOverridesDecisionsPackTest.php` covers export, replay with people and dates kept, the tie-out, and a second replay that changes nothing.
- `tests/Feature/Eir/LoanAgeingOverlayTest.php` and `LoanGrossAmountOverlayTest.php` cover the overlays.

9. What is still owed

1. **MAIIC's facility-level reasons**, due Monday 19 October 2026: 36 for staging and 1 for MUSECO. Each reason replaces the audit-approval wording through a new proposal and approval. The old override is withdrawn, not edited.
2. **A second person's approval.** All 37 were self-approved under UAT mode. Before December 2025 is relied on outside UAT, a second person should review and approve them, or withdraw and re-propose them with MAIIC's reasons for a second person to approve.
3. **Golden numbers.** The overrides change December 2025's figures, so the golden values for that month no longer hold. Resetting `docs/bootstrap/golden.json` needs Edward's written approval, with the before and after shown.

4 **Differences not overridden**, on purpose: - 3 facilities in a different arrears bracket but the same stage; - 1 workbook facility with no loan in the system; - 9 system loans that are not in the workbook.

They are listed on `staging:december-check`'s CSV with their causes.